LAB 4

AI-Assisted Development Chat History & Work Summary

Connect Four — Win Detection, Game Flow, and AI Move Selection

Student SRN	PES1UG24AM139
Project repository	15_connect_four
Source repository	github.com/SETAPESU26/15_connect_four (assigned repository)
Record type	Curated summary of the AI-assisted workflow

Purpose. This document organizes the main questions, development changes, and verification steps from the Connect Four assistance session into a readable submission record. It is a curated summary, not a verbatim export of every chat message.

Prepared for Lab 4 submission

1. Objective and Scope

The task was to improve an existing Python Connect Four game. The work focused on correctly detecting four-in-a-row wins, making AI moves safer and more useful, handling invalid or unavailable moves, and ensuring the game stops immediately after a win or draw.

Files involved

- **board.py** — board representation, disc placement, full-board detection, and win detection.
- **ai.py** — AI legal-move selection, immediate winning moves, and blocking moves.
- **game.py** — turn flow, user input validation, move feedback, and game termination.
- **main.py** — application entry point; left unchanged.
- **test_board.py** — automated checks added for board logic and AI behaviour.

2. Main Chat Topics and Decisions

Topic A — Win detection

The original board logic checked horizontal and vertical directions only. The updated logic checks four directions: horizontal, vertical, diagonal down-right, and diagonal down-left. A win is reported only when four matching tokens are found in a valid line.

Topic B — AI move selection

The AI first checks whether it can win immediately. If not, it checks whether the player has an immediate winning move and blocks it. If neither condition applies, it chooses a legal column closest to the centre. The AI checks available columns before selecting a move.

Topic C — Game flow and input handling

The game validates user input, rejects column numbers outside 1–7, handles full columns, and supports quitting with q. After a successful disc placement, the game prints feedback once. It checks for a win before switching to the next turn and checks for a draw when the board is full.

Topic D — Testing and Git workflow

A separate test script was added to exercise win detection, invalid/full-column moves, and AI move selection. The changes were committed to the local Git repository. The user reported that Git later displayed a clean working tree with nothing left to commit.

3. Implementation Summary

Board logic

- Added all four directional checks for four-in-a-row detection.
- Kept bounds checks so diagonal searches cannot read outside the board.
- Made disc placement reject out-of-range or full-column moves by returning no row.

AI logic

- Collects only columns that are not full.
- Temporarily tests candidate moves for an immediate AI win, then restores the board.
- Temporarily tests candidate moves for an opponent win and blocks when needed.
- Returns no move when there are no legal columns.

Game loop

- Validates typed input and handles non-numeric input without crashing.
- Prints move feedback only after a disc is successfully placed.
- Stops immediately on a win, a full-board draw, or a quit command.

4. Verification Results

The user ran `python test_board.py` and reported that all automated tests passed. The checks included:

Test	Expected result	Reported
Horizontal win	Detected	PASS
Vertical win	Detected	PASS
Diagonal down-right win	Detected	PASS
Diagonal down-left win	Detected	PASS
Three tokens in a row	Not a win	PASS
Full column	Reject additional disc	PASS
Out-of-range columns	Reject invalid columns	PASS
AI immediate winning move	Choose winning column	PASS
AI blocks immediate player win	Choose blocking column	PASS
No legal AI moves	Return no move	PASS

Scope note: The automated test script directly checks board win detection, full/out-of-range columns, and key AI decisions. Interactive input cases such as typing letters, quitting, and end-to-end video behaviour should also be demonstrated or manually checked where required by the lab rubric.

5. Refined Chat History

The following entries summarize the main requests and outcomes from the assistance session. They are rewritten for clarity and are not exact quotations.

1. Diagnose the assignment

Reviewed the Connect Four task requirements and identified the key areas to improve: win detection, draw/full-column handling, AI winning/blocking, feedback, and termination.

2. Fix board win detection

Updated board logic to search horizontally, vertically, and along both diagonal directions. Added checks for shorter sequences and invalid columns.

3. Improve the AI

Changed AI selection to take an immediate win when available, block an immediate opponent win, avoid full columns, and otherwise prefer a central legal column.

4. Correct the game loop

Added input validation, quit handling, full-column handling, one feedback message per successful placement, and win/draw checks before the next turn.

5. Add automated tests

Created test_board.py and ran it. The user reported that the horizontal, vertical, diagonal, three-in-a-row, full-column, invalid-column, and AI tests all passed.

6. Save the code changes

Reviewed the Git status and commit/push workflow. The user later reported a clean working tree with nothing left to commit.

7. Prepare submission materials

Planned a Lab-4 folder containing before/after gameplay videos and the updated project files. This PDF organizes the AI-assisted work summary for documentation.

6. Submission Checklist

- Before video — demonstrates the original behaviour or issue, if captured.
- After video — demonstrates the updated game running and showing move feedback.
- Updated code — board.py, ai.py, game.py, main.py, test_board.py, and relevant project files.
- This refined AI chat-history summary PDF.

Before submitting, confirm that the Lab-4 folder is in the correct Lab repository and that the videos and code files have actually been uploaded. This document does not itself upload files to GitHub.

End of report